TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REGISTER

Class: **Duration time**: from 09/2026 To 03/2027

(*) Profession: Software Engineer **Specialty**: SE [x]

(*) Kinds of person make registers: Lecturer [x] Students

1. Register information for supervisor (if have)

No.	Full name	Phone	E-Mail	Title
Supervisor	Đặng Ngọc Minh Đức		ducdnm2@fe.edu.vn	Giảng viên
Co-supervisor	Bùi Văn Phúc			Giảng viên

2. Register information for students (if have)

No.	Full name	Student code	Phone	E-mail	Role in Group
1	Đỗ Đăng Khoa	SE192357		khoado7577@gmail.com	Leader
2	Trần Khải Hoàng	SE182767		HoangTKSE182767@fpt.edu.vn	Member
3	Lâm Phi Long	SE182769		LongLPSE182769@fpt.edu.vn	Member
4	Nguyễn Bá Tân	SE183860		TanNBSE183860@fpt.edu.vn	Member
5	Nguyễn Ngọc Long	SE193490		nguyenngoclong216@gmail.com	Member

3. Register content of Capstone Project

(*) 3.1. Capstone Project name:

3.1.1. English: ***TrekLink: An Integrated Off-Grid Communication and Smart Device Rental Management Platform for Trekking Agencies***

3.1.2. Vietnamese: ***TrekLink: Nền tảng tích hợp liên lạc ngoài vùng phủ sóng và quản lý cho thuê thiết bị thông minh cho doanh nghiệp trekking***

Abbreviation: TrekLink

(*) 3.2. Main proposal content (including result and product)

a) Context:

The problem. Trekking companies lead groups along mountain routes where mobile phone coverage is weak or missing for long stretches. Satellite messengers need both the device and

a paid satellite subscription [14], so they cost more to buy and run, and few groups carry them. In Vietnam, the market for trekking and off-grid communication systems is still new and has room to grow.

The solution. TrekLink builds LoRa mesh devices and rents them by plan to organizations through a single platform that TrekLink hosts. An SOS alerts the organization's own on-duty staff first and goes to TrekLink Staff when nobody answers. The team's Meshtastic-based firmware [6] relays messages over several hops, and the platform adds live monitoring.

In Summer 2026 the project team developed TrekLink, an open-source LoRa mesh communication firmware for ESP32/ESP32-S3 devices. The team wrote the firmware on top of Meshtastic, and it supports multi-hop mesh messaging, GPS tracking, fall detection and automatic SOS broadcasting. The firmware is kept under version control and is the technical foundation that this Capstone inherits. The Capstone does not rebuild it.

TrekLink is a smart-device management platform whose customers are trekking companies. It does not manage tours. TrekLink owns the devices, the firmware, the Field Station software and the cloud platform, and rents the devices to organizations by plan. Many organizations share the one platform. Each of them manages its own members, on-duty roster, rented devices, alerts and API keys on it, and keeps its trips, customers and emergency response outside it.

Scope disclaimer. TrekLink is a supplementary safety and coordination tool with no guaranteed rescue link. A device works only within LoRa range of another TrekLink node, a relay checkpoint or the Field Station. The Field Station reaches the cloud only while it has an internet connection. The device fixes its position by GPS and shares it over LoRa only, with no satellite uplink. A device out of range keeps beaconing, and its own durable queue delivers the stored events once a node hears it. When the battery runs out, the organization receives a stale-device alert with the last known position and trail, and no further data. When no node hears an SOS, the device repeats it until one does, and the platform flags a suspected episode from the beacon pattern. Range drops sharply in heavy rain, behind a ridge with no relay on the crest and through thick concrete.

Scope Boundary: The LoRa mesh firmware and the hardware that this team built in Summer 2026 are the technical foundation. Targeted firmware enhancements made during the Capstone, such as the durable on-device queue, the SOS beaconing fixes and radio power compliance, are in scope and credited to the Capstone. Rearchitecting the mesh stack is out of scope. The Capstone covers the Field Station gateway bridge, offline-resilient synchronization, the cloud backend, organization and asset management, rental contracts, incident management with tiered alerts, live telemetry with an organization API, billing, system integration and experimental evaluation.

The Capstone serves the following needs:

- **Trekkers and guides** need to share their position, exchange preset messages and raise an alert from the device itself, without a phone, in places a phone cannot reach.
- **Trekking companies** need to see their own groups and devices and to receive alerts that have a clear owner and a record, without buying or maintaining the hardware.

- **Local authorities and rescue units** need a timely report with the last known position and the signal history when an incident escalates.
- **TrekLink, as the provider,** needs to track the custody, condition and charges of each rented device across many renters, and to deliver field events through long outages.

b) Proposed Solutions:

TrekLink extends the existing LoRa mesh firmware into an enterprise device-rental platform built on asset management. The platform has six core capabilities:

- **Organization and rental management:** Organizations register themselves, and TrekLink verifies and approves them. They rent devices on a monthly plan or a day plan, with a minimum order quantity. On the monthly plan, half the term fee is paid at the start as a holding fee and half at the end, and the contract renews until the organization gives notice. A day plan lasts 3, 4 or 7 days, is paid in full at handover and costs more per day than the monthly rate. Handover and return take place in person at the TrekLink counter with a signed handover note. Payments are recorded online through a third-party payment gateway, VietQR in its sandbox, or at the counter. Each organization sorts the devices it holds under all its contracts into its own groups with one level of subgroups, for example one group per site with smaller teams inside it, and labels each device with the name of its holder. TrekLink does not model trips or tours, so each organization uses groups for its own purpose. The organization's staff can also place a rented node on site as a temporary relay checkpoint and retrieve it later. Relays belong to the same contract, and a node left on site after the contract ends counts as lost.

The rental plans and charges below are the default values. A TrekLink Admin can change each of them.

Item	TrekLink v2	TrekLink v3	TrekLink v4
Monthly plan, per device per month	400,000 VND	450,000 VND	480,000 VND
Day plan, per device per day (monthly ÷ 30 × 1.5)	20,000 VND	22,500 VND	24,000 VND
Day-plan lengths	3, 4 or 7 days	3, 4 or 7 days	3, 4 or 7 days
Minimum order per contract	5 devices	5 devices	5 devices
Late return, per device per day after the grace period	50,000 VND	50,000 VND	50,000 VND
Damage	by inspection; above 500,000 VND needs Admin approval	same	same
Loss	remaining value of the device	same	same

A monthly contract opens with a holding fee of half the term fee, and each term closes with the other half. The contract renews automatically, the organization can give notice during a running term, and the contract finally closes after return and inspection. A day-plan contract is paid in full at handover and has a due date. A late fee is charged per day after the grace period, and the contract closes after return and inspection.

- **Asset management:** Each physical device carries two identities, a printed asset tag and its radio node number, and moves through a nine-state lifecycle: In Intake, Available, Reserved, Rented, Returned, Maintenance, Lost, Retired and Disposed. On the main path, an Available device becomes Reserved when a contract is approved, Rented at handover and Returned at check-in. A Returned device goes back to Available after a reset and a passed inspection, or to Maintenance after a failed one. A device in Maintenance becomes Available after repair or is Retired, and a Retired device becomes Disposed after a verified wipe. A loss can be reported from any of the six working states. It opens a loss case, which ends either with the device recovered to Returned or with the loss settled and the device Retired. The renter bears a loss only when the device was in its custody. At handover each device is provisioned with the renter's channel key, and on return it is reset and inspected before it can be rented again.
- **Offline-Resilient Field Station:** Each field event is identified by eventId = sha256(nodeNum : packetId), which is derived from identifiers the firmware already transmits. The Field Station is a single executable that the organization runs on its own laptop, with one rented node on a USB port, at a fixed base point of its choice such as a basecamp, a vehicle, a ranger station or a trailhead. It uploads over whatever internet connection it finds there, site Wi-Fi or a phone hotspot. When the internet is unavailable, the Field Station stores events persistently in a local SQLite queue and shows the local mesh to the operators on site. SOS events are safety-critical and get priority P0, so they are synchronized first on reconnection. GPS updates are P2 and routine telemetry is P3. Each device also keeps its own durable queue. An event records when it occurred and the source of that time, and it belongs to the organization that held the device at that moment. An event that synchronizes after the device was returned and rented again therefore reaches the earlier renter and never the new one.
- **Idempotent Event Processing:** The cloud backend uses the Event ID as an idempotency key [9]. The same event may arrive several times through retransmission, replay after reconnection or a duplicate LoRa broadcast, and the backend still creates exactly one Incident and one alert per tier. This prevents alert fatigue and keeps the data consistent.
- **Incident Management with tiered alerts:** Each SOS episode becomes one incident, routed to the organization that holds the device. Alerts go to the organization's on-duty member, then to the backup and then to the Manager, each after a timeout. If nobody acknowledges, the incident escalates to TrekLink, which reports it to the authorities. The organization runs the response and reports its status back. Each transition is atomic and is audited with the actor, the time and the reason.
- **Live telemetry:** Each organization sees only its own rented devices, positions, battery levels and alerts on a live map. It can read the same data in its own systems through a REST and WebSocket API with an organization key.

Technology stack:

- **Embedded firmware:** ESP32/ESP32-S3 with the TrekLink firmware built by the team, with targeted enhancements in scope

- **Field Station:** Node.js (TypeScript) packaged as a single executable with a local web page: async I/O for concurrent MQTT and LoRa serial communication
- **Backend:** NestJS (TypeScript): modular architecture with REST and WebSocket support
- **Database:** PostgreSQL with Prisma ORM: ACID compliance for rental transactions and the incident audit trail
- **IoT messaging:** MQTT (Mosquitto): QoS-based delivery between the Field Station and the backend [4]
- **Real-time push:** Socket.io WebSocket: live dashboard updates and the organization API stream
- **Frontend:** React and TypeScript: role-based views with Leaflet.js map integration
- **Payment gateway:** a third-party payment gateway, VietQR with a confirming webhook, in its sandbox environment
- **DevOps:** Docker Compose and GitHub Actions: containerised deployment and CI/CD

c) Functional Requirements:

- **TrekLink Admin:**

• Approve or reject organizations, and suspend or reactivate them.

• Manage platform accounts, roles, permissions and system configurations.

• Manage TrekLink hardware variants, firmware versions and their remaining-value schedules.

• Manage rental plans and prices: the monthly price per variant, the day-plan lengths and premium, the minimum order quantity, and the late, damage and loss charges.

• Approve damage charges above the threshold, except for an inspection the Admin performed, and retire devices.

• Monitor system health, Field Station connectivity, alert delivery and incidents; view audit logs and management reports.

- **TrekLink Staff:**

• Verify organizations in person or by phone and record the signed master contract, which includes the non-disclosure agreement and the Field Station software licence. Record suspected fraud, theft, IP breaches and NDA breaches as compliance cases.

• Register devices with their asset tag and node number, run intake checks and record maintenance.

• Approve rental contracts and reserve devices for them without over-allocation.

• Provision devices with the organization’s channel key and hand them over at the counter with a signed handover note.

• Check devices in, reset and inspect them, record damage and loss, and close contracts.

• Record counter payments and run stock-takes.

• Handle escalated and unrouted incidents and record reports to the authorities.

- **Organization Manager:**

• Register the organization and sign the master contract and the handover notes.

• Request rental contracts with a plan, hardware variant, quantity and start date, pay online or at the counter, and give notice on monthly contracts.

• Manage the organization's members, its on-duty roster with a primary and a backup per shift, and its API keys.

• View contracts, terms, invoices and payments, and act as the last organization alert tier.

• Manage the organization's rented devices across its contracts in one console that shows each device's variant, asset tag, status, battery, last-seen time, group and holder. Create groups and one level of subgroups, assign devices and holder names, report a fault or a loss, and give return notice.

- **Organization Operator (on-duty member):**

• Receive SOS alerts, acknowledge incidents, report the response status and outcome, or declare a false alarm.

• View the organization's rented devices, positions, battery levels and alerts on the live map.

• Run the Field Station at the base point and view the local mesh offline; view devices by group and holder.

- **Organization System (API client):**

• Read the organization's live positions, telemetry and incident events over REST and WebSocket with an organization API key.

- **Asset Management:**

• Register each physical unit with its asset tag, node number, hardware variant, firmware version and status. Neither identity is ever reused.

• Manage the nine-state device lifecycle of In Intake, Available, Reserved, Rented, Returned, Maintenance, Lost, Retired and Disposed through its allowed transitions only. For example, a returned device goes back to Available after a passed inspection, and a device in a working state can be reported Lost.

• Handle each lost device as a loss case. Custody is decided from the rental history and channel key exposure is flagged. The case ends in recovery or in a charge according to custody, which an Admin can waive, and suspected theft or fraud carries a police report reference. Replace a faulty or lost unit on a running contract at the counter.

• Record the battery level, last-seen time, latest GPS position and telemetry status.

• Maintain the custody, contract, incident, inspection and maintenance histories, and reset each returned device before it can be rented again.

• Schedule maintenance from a service plan per hardware variant and a weekly scan. Record each maintenance cost, with Admin approval above a threshold, and each firmware update. Prevent unavailable or due-for-service devices from being allocated.

- **Event Reliability & Offline Synchronization:**

• Each field event shall contain an eventId derived as sha256(nodeNum : packetId), the source device ID, the event type, a UTC timestamp and a priority tier.

• The Field Station shall persist unsent events in a local SQLite queue when cloud connectivity is unavailable, and each device shall keep its own durable queue.

• SOS and other safety-critical events (P0) shall receive a higher synchronization priority than GPS updates (P2) and routine telemetry (P3). On reconnection, P0 events shall be delivered before any lower-priority queued events, regardless of the order in which they arrived.

• The Gateway shall flush queued events automatically after connectivity is restored and report the synchronization status, retry count and queue depth per priority tier.

• The backend shall process all incoming events idempotently, with eventId as the idempotency key.

• Duplicate reception or retransmission of the same SOS event shall create exactly one Incident and one logical notification event in the backend.

• The system shall record the synchronization status, retry count and processing history of each event for diagnosis and audit.

• Acceptance criterion: repeated delivery of the same SOS event, with an identical eventId, shall produce exactly one Incident record and one notification, however many times it is delivered.

- **Live Telemetry & Incident Management:**

• Display each organization’s rented devices, positions, device status and last-seen information on a real-time map, scoped to that organization on the server, and offer the same data through the organization API.

• Create an Incident automatically when a valid SOS event is received and passes the idempotency check, and route it to the organization that holds the device.

• Alert the organization’s on-duty member within 2 seconds of incident creation, then the backup and the Manager after configurable timeouts, and escalate to TrekLink when nobody acknowledges.

• Support the incident lifecycle with tiered notification, acknowledgement, response, resolution, false alarm, escalation, authority report, reopening and closing.

• Record the actor's user ID and role, a timestamp and an action note at each state transition.

• Keep a complete, append-only incident history for audit.

• Keep each incident owned at all times, by a named owner or by a running tier timeout.

- **Billing & Reporting:**

• Calculate monthly term fees, made up of the holding fee and the balance, and day-plan fees per device and hardware variant, together with late, damage and loss charges.

• Generate invoices and track payment status.

• Integrate a third-party payment gateway, VietQR with a confirming webhook, in its sandbox environment, and record counter payments.

• Generate reports on contracts, fleet utilization, revenue, overdue and lost devices, maintenance, incidents and response performance.

d) Non-Functional Requirements:

- **API Performance:** At least 95% of normal backend API requests shall complete within 300 ms under the defined test workload of 50 concurrent users.
- **Synchronization Latency:** When gateway uplink connectivity is available, newly received field events shall reach the cloud backend within 5 seconds under normal load.
- **Offline Recovery: Delivery Rate:** The Gateway shall achieve a message delivery rate of ≥ 99% after connectivity recovery under the defined experimental conditions: connectivity-loss durations of 30 s to 30 min and at least 20 trials per condition.
- **Offline Recovery: Testability:** Connectivity loss shall be reproducible in a controlled lab environment by programmatically interrupting the gateway's network interface, so that experiments can be repeated.
- **Duplicate Prevention:** Repeated delivery of an SOS event with the same eventId shall create zero duplicate Incident records. A dedicated automated test shall verify this by delivering the same SOS event 10 times and asserting that exactly one Incident is created.
- **Priority Ordering:** On each reconnection, all P0 (SOS) events queued during the offline period shall be delivered to the backend before any P2 or P3 events. The priority-ordering compliance rate shall be ≥ 99% across all experimental trials.
- **Concurrency:** The backend shall process at least 20 simultaneous gateway event submissions without message loss or duplicate Incident creation, verified by a concurrent load test.
- **Offline Resilience:** Loss of cloud connectivity shall not interrupt local LoRa mesh communication or device-level SOS propagation on the field devices.
- **Data Consistency:** Replayed gateway messages shall not create duplicate incidents or duplicate telemetry records. Idempotency keyed on eventId at the backend enforces this.

- **Security:** JWT-based authentication, RBAC, TLS-encrypted transport, bcrypt password hashing and auditable access to sensitive incident data. No organization shall be able to read another organization's devices, positions, incidents, contracts or invoices, and a cross-organization test per module shall verify this.
- **Auditability:** Each incident state transition, device lifecycle transition and rental contract event shall be traceable to an actor, identified by user ID and role, and to a UTC timestamp.
- **Scalability:** The system shall support at least 10 organizations and 50 rented TrekLink devices concurrently without architectural changes.
- **Deployability:** All server components shall support containerized deployment with Docker Compose and automated CI/CD with GitHub Actions.

e) Theory & Practical:

Theory: The project applies software engineering and distributed systems principles to integrate an intermittently connected LoRa mesh network with a cloud-based operational platform.

- **Finite State Machines:** Model the nine-state device lifecycle with its loss case, the incident lifecycle with tiered notification and escalation, and the organization and rental-contract lifecycles as state machines with controlled, auditable transitions [8]. Each transition is an atomic, logged operation.
- **Distributed Systems and Eventual Consistency:** MQTT loosely couples the Gateway and the cloud backend. Field events are produced offline and delivered when connectivity allows, and the backend converges to a consistent state once all queued events are processed [7].
- **Priority-Aware Store-and-Forward:** Classify field events by safety criticality into P0 SOS, P1 incident-related location, P2 GPS tracking and P3 routine telemetry, and keep that order during the queue flush so that safety-critical events are not delayed behind routine data [5].
- **Idempotent Event Processing:** Backend event handlers produce the same outcome however many times an event is delivered, with eventId as a natural idempotency key [9]. Reliability is therefore handled above the transport layer, without relying on exactly-once delivery from MQTT.
- **IoT Messaging Protocols:** MQTT QoS 1 delivers each message at least once [4]. Combined with idempotency in the backend, it gives effectively exactly-once processing without QoS 2.
- **Event-Driven Architecture:** An SOS event starts an asynchronous pipeline of event ingestion, idempotency check, incident creation, role-based notification and audit logging. The stages are decoupled from each other and can be tested independently.
- **Role-Based Access Control (RBAC) and multi-tenancy:** The API layer enforces fine-grained permissions for TrekLink Admin and Staff and for each organization's Manager and Operators, and the server isolates each organization's data.
- **Software Architecture:** A domain-driven modular design separates gateway synchronization, organizations, devices, rental contracts, incidents, monitoring, billing

and platform services into independent NestJS modules with documented REST and WebSocket APIs.

Practical: Build on the team's Summer 2026 ESP32/ESP32-S3 LoRa mesh firmware and hardware, with targeted enhancements. Document the complete message schema, covering the eventId structure, message types and payload format, before gateway implementation begins.

- Develop the Field Station in Node.js and TypeScript: the LoRa-to-serial message parser, SQLite persistent priority queue, MQTT uplink publisher, reconnection flush logic, Field Station health reporter, local web page and single-executable packaging.
- Develop the NestJS backend: the auth and RBAC module, organizations module, asset (device) module, rental contract module, gateway sync module for idempotent event ingestion, incident module with tiered alerts and WebSocket notifications, billing module with the payment gateway, and reporting module. Apply the PostgreSQL migrations with Prisma.
- Develop the React and TypeScript web client: role-based views for TrekLink and for organizations, a real-time operational map built with Leaflet.js, an incident dashboard with WebSocket updates, contract and asset management interfaces, the organization API and reporting views.
- Deploy with Docker Compose, and automate testing and deployment with GitHub Actions CI/CD.
- Design and run the experimental protocol for RQ1, RQ2 and RQ3 as specified in section 3.3.

f) Products (Expected Deliverables):

- **TrekLink Rental Platform:** Integrated web application for TrekLink Admin and Staff and for each organization's Manager and Operators.
- **Field Station:** LoRa-to-cloud gateway run on the organization's laptop, with a persistent SQLite priority queue for tiers P0 to P3, idempotent MQTT synchronization, reconnection flush, health reporting, and a local offline view. It is the main new hardware-integration deliverable.
- **Event Reliability Layer:** The end-to-end eventId scheme, backend idempotency enforcement, synchronization audit log and duplicate-prevention test suite.
- **Organization & Rental Contract Module:** Organization registration and approval, members, on-duty roster, API keys, monthly and day plans, handover, terms, notice, overdue and default.
- **Asset Management Module:** Nine-state device lifecycle, intake, provisioning, reset, inspection, scheduled maintenance with cost approval, loss cases, replacement, disposal and stock-take.
- **SOS Incident Management Module:** Automatic Incident creation from SOS events, tiered alerts with escalation to TrekLink and authority-report records, WebSocket notification, response tracking and append-only audit history.
- **Live Telemetry Module:** Organization-scoped live map, device telemetry, positions, Field Station connectivity, and a REST and WebSocket API per organization.

- **Billing Module:** Term and day-plan pricing, late, damage and loss charges, invoices, and online payments through the payment gateway in its sandbox.
- **Backend API & Real-Time Services:** A REST API documented with OpenAPI/Swagger, and Socket.io WebSocket services.
- **Technical Documentation:** SRS with UML state machine diagrams for the Device, Incident, Organization and Rental Contract, system architecture, database schema, API reference, test plan, deployment guide and user guide.
- **Evaluation Report:** Experimental results for RQ1 on gateway delivery rate, latency and duplicate rate, for RQ2 on the effect of connectivity-loss duration, and for RQ3 on the SOS-to-Incident workflow against the baseline.
- **Source Code & Deployment Package:** Version-controlled repositories for the gateway, backend and frontend, the Docker Compose configuration, GitHub Actions CI/CD and a runnable demonstration environment.

g) Proposed Tasks:

- **TP1: Requirements, Architecture & Experimental Design (Weeks 1–3):** SRS, UML state machine diagrams for the Device Lifecycle and the Incident Lifecycle, the system architecture document, an analysis of the inherited components covering the firmware message schema and eventId structure, database schema design, the RQ and experiment protocol design, and a proof-of-concept LoRa-to-Gateway serial message parser. All later TPs depend on the frozen message schema produced in TP1.
- **TP2: Gateway Bridge & Offline Synchronization (Weeks 2–6):** LoRa-to-Gateway serial integration, the SQLite persistent priority queue with tiers P0 to P3, MQTT publish logic, reconnection detection and the flush routine, duplicate prevention by eventId at gateway level, the gateway health monitoring API, and unit and integration tests for the offline queue, priority ordering and reconnection recovery.
- **TP3: Core Backend & Reliability Services (Weeks 3–7):** The NestJS authentication and RBAC module, device fleet FSM module, rental lifecycle module, idempotent event ingestion endpoint with eventId-keyed deduplication, synchronization audit log, PostgreSQL schema and migrations, and automated tests for idempotency, where 10 deliveries of one eventId create 1 Incident, and for concurrent event submission, where 20 simultaneous requests create 0 duplicates.
- **TP4: Monitoring & Incident Management (Weeks 5–9):** Real-time telemetry ingestion, an organization-scoped Leaflet.js map with WebSocket live updates, the organization telemetry API, the SOS-to-Incident processing pipeline, the tiered-alert incident FSM with escalation, acknowledgment workflows and the append-only incident audit trail.
- **TP5: Business Operations (Weeks 6–10):** Organization onboarding, rental contracts and plans, device reservation, handover and check-in at the counter, reset and inspection, billing with online payments, TrekLink and organization web views, operational dashboards, audit log views and management reports.
- **TP6: Integration, Experimental Evaluation & Deployment (Weeks 9–13):** Physical LoRa experiments for RQ1 and RQ2 with 3–5 devices, the connectivity-loss matrix and ≥ 20 trials per condition; offline and recovery experiment results, duplicate-event test results and concurrency and load test results; cross-organization isolation suite results

with at least two organizations per module, and simulated multi-organization load test results; the RQ3 SOS drill evaluation in randomized order with ≥ 15 drills per condition and automatic timestamp recording; the evaluation report, end-to-end integration test suite, Docker Compose deployment, GitHub Actions CI/CD pipeline and final Capstone documentation.

Risk assessment:

- **Insufficient physical TrekLink devices** (Likelihood: Medium / Impact: High): Simulate additional Gateway-to-Cloud load with MQTT scripts. Physical devices are needed only for the mesh-to-gateway experiments, and simulation shall not be used to claim LoRa RF reliability.
- **Gateway sync latency exceeds 5-second NFR** (Likelihood: Medium / Impact: Medium): Tune the MQTT QoS level, reduce the payload size and apply backpressure to the queue flush rate.
- **RQ3 baseline not objectively measurable** (Likelihood: High / Impact: Medium): Use randomized simulation drills in which an independent observer triggers the SOS and records the timestamp, and participants do not know the exact trigger time in advance.
- **NestJS + MQTT + WebSocket integration underestimated** (Likelihood: Medium / Impact: High): Complete a proof-of-concept gateway-to-backend integration in TP1 Week 2 instead of waiting for TP3.
- **Firmware message schema incompatible with new gateway** (Likelihood: Low / Impact: High): Freeze and document the schema in TP1 before gateway implementation begins.
- **Scope creep** (Likelihood: High / Impact: Medium): Freeze features after TP5 Week 10 and move additional features to the post-Capstone backlog.

3.3. Research Information

a. Research Problem / Research Question

Trekking companies and similar organizations that operate where cellular coverage is limited need field events, including SOS alerts, to reach their staff even after a long outage. They also need each SOS to be handled as a coordinated incident with an owner, a tracked acknowledgment and an audit trail.

The Summer 2026 TrekLink firmware provides the field communication layer. This Capstone adds a Field Station with priority-aware offline queuing and idempotent event processing, and an SOS-to-Incident pipeline with a formally defined lifecycle and tiered escalation. It then evaluates both.

The evaluation answers three research questions:

- **RQ1:** How reliably can the TrekLink Gateway Bridge synchronize SOS, GPS, and telemetry events from an intermittently connected LoRa mesh to the cloud backend, measured by message delivery rate, data-loss rate, and duplicate rate?

- **RQ2:** How do varying connectivity-loss durations and reconnection patterns affect message delivery rate, synchronization latency, and priority-ordering compliance of the proposed Gateway architecture?
- **RQ3:** Compared with the existing uncoordinated device-level SOS workflow, to what extent does the TrekLink SOS-to-Incident pipeline improve incident response coordination, traceability, and acknowledgment consistency, as measured by MTTA, MTTR, traceability score, and incident-completion rate in controlled emergency drills?

RQ1 and RQ2 evaluate the Event Reliability & Offline Synchronization functional requirements. RQ3 evaluates the Incident Management functional requirements and the incident FSM. The three RQs trace back to the functional requirements, system modules, task packages and experiment conditions defined in this proposal.

Baseline definition for RQ3: In the existing workflow, a TrekLink device triggers an SOS broadcast across the LoRa mesh. If a guide's Meshtastic-paired phone receives the alert, the guide contacts staff by voice call or a messaging application. The workflow has no structured acknowledgment, timestamp recording, response-action log or audit trail. Baseline MTTA is the time from the SOS trigger on the device to the first documented verbal or text acknowledgment, recorded by an independent observer. The baseline traceability score is 0% by definition, because no audit record exists. The study does not assume in advance that MTTA will fall, and the outcomes will be reported as measured.

b. Research Objectives

- Design and implement a priority-aware, offline-resilient Gateway Bridge with end-to-end idempotent event processing that connects the TrekLink LoRa mesh to a cloud backend.
- Evaluate gateway synchronization reliability under a controlled matrix of connectivity-loss durations and reconnection patterns, using physical TrekLink hardware for all mesh-to-gateway measurements.
- Design and implement an SOS-to-Incident pipeline with a formally defined FSM, tiered notification and escalation, and an append-only audit trail.
- Evaluate the efficiency of the incident-response workflow by MTTA, MTTR, traceability score and incident-completion rate in randomized controlled emergency drills, against the uncoordinated baseline.
- Integrate the evaluated gateway and incident mechanisms into a complete multi-organization rental platform covering asset management, rental contracts, live telemetry and billing.

c. Research Scope & Methodology

In scope: TrekLink ESP32/ESP32-S3 devices and the LoRa mesh running the Summer 2026 firmware with targeted enhancements; the Field Station with LoRa-to-MQTT-to-cloud synchronization, a priority queue and idempotency; the SOS, GPS and selected telemetry message types; connectivity-loss and recovery experiments in a controlled lab environment; evaluation of the SOS-to-Incident workflow in controlled drills; the organization, asset and rental-contract modules with multi-tenancy; a responsive web client for desktop and tablet; payment in the sandbox only.

Out of scope: mesh-stack rearchitecture, native mobile apps, commercial payment, hardware radio certification, individual renters, and the organizations' own trip and rescue operations.

Methodology:

- **Literature Review:** Survey LoRa mesh systems [1], store-and-forward and DTN protocols [5], IoT fleet management and emergency incident workflows [3], and identify the integration gap between them.
- **Architecture & Design:** Freeze the firmware message schema and eventId structure. Design the gateway synchronization architecture, SQLite priority queue schema, idempotency table, incident FSM and experiment protocol. Produce the SRS, UML state machine diagrams, architecture diagrams and sequence diagrams.
- **Implementation:** Develop the gateway bridge, backend modules and web client according to TP1 to TP5.
- **Experiment 1: Gateway Synchronization (RQ1 & RQ2):** Use 3–5 physical TrekLink devices for all mesh-to-gateway reliability measurements. MQTT simulation scripts may add Gateway-to-Cloud workload only and are not used to claim LoRa RF reliability. Apply a connectivity-loss matrix of 0 s as the control, 30 s, 2 min, 5 min, 10 min and 30 min, with two reconnection patterns, immediate and delayed by 60 s. Run at least 20 trials per condition. Measures: delivery rate (%), data-loss rate (%), duplicate rate (%), sync latency (mean / median / P95 in seconds), queue-recovery time (s) and priority-ordering compliance rate (%).
- **Experiment 2: SOS-to-Incident Workflow (RQ3):** Run controlled emergency drills with two organizations at once, each with 1 TrekLink device, 1 Field Station, an on-duty member and a backup, and at least 4 participants in rotation. The scenario order is randomized. Participants do not know the exact SOS trigger time. An independent observer triggers the SOS and starts an external timer. The backend records all timestamps in the TrekLink condition automatically, and the observer records the baseline timestamps with a hardware stopwatch. Run 15–20 drills per condition and vary the network latency between LAN and a 4G hotspot. Measures: MTTA (mean / median / P95 in seconds), MTTR (mean / median in minutes), traceability score (% of FSM transitions with complete actor, timestamp and action), missed-alert rate (%), duplicate-alert rate (%), incident-completion rate (%), and cross-organization leakage, meaning alerts or positions delivered to the wrong organization, expected to be 0.
- **Integration & System Testing:** End-to-end test cases for all functional requirements; idempotency tests in which 10 deliveries of one eventId create 1 Incident; concurrency tests in which 20 simultaneous events create 0 duplicates; RBAC validation; an audit-log completeness check; a cross-organization isolation suite with at least two organizations per module, including an event that synchronizes after its device was returned and rented to another organization; and a load test with simulated devices and Field Stations at 10 organizations, 50 rented devices and 20 simultaneous Field Station submissions.

d. Expected Scientific Contribution

- **A priority-aware, offline-resilient gateway pattern** for connecting intermittently connected LoRa mesh devices to cloud-based operational systems, with a formally

defined eventId scheme, a SQLite persistence model and idempotent backend processing.
- **An empirical benchmark** of gateway synchronization reliability and latency under a controlled connectivity-loss matrix on low-cost ESP32 hardware.
- **An SOS-to-Incident workflow design** with a formally specified FSM, tiered notification and escalation chain, and an append-only audit trail, traceable to measurable research outcomes.
- **A comparative evaluation** of uncoordinated and structured incident response by MTTA, MTTR and traceability in randomized controlled drills.
- **A practical case study** of the end-to-end integration of an open-source LoRa mesh prototype into a multi-organization rental platform for trekking agencies.

e. Related Works / Literature Review (Preliminary)

- **LoRa mesh communication** (Meshtastic, RAK WisBlock Mesh) [1], [2], [6]: multi-hop messaging, GPS, local SOS broadcast and fall detection. These systems are designed to work without any infrastructure. TrekLink builds on them and adds the organization-side layer of device rental and fleet management, store-and-forward to the cloud and an incident pipeline.
- **IoT fleet management platforms** (AWS IoT Core, Azure IoT Hub, Balena) [12]: device registration, telemetry ingestion, remote monitoring and lifecycle states. Gap: they are generic cloud platforms with no LoRa mesh integration, no store-and-forward for intermittent connectivity and no rental domain.
- **Delay/Disruption-Tolerant and store-and-forward systems** (DTN RFC 4838 Bundle Protocol, MQTT-SN, LoRaWAN ADR) [5], [10], [11]: message persistence and forwarding under intermittent connectivity. Gap: they are protocol-level designs and are not integrated with business workflows or incident management.
- **Incident management systems** (JIRA Service Management, PagerDuty, Opsgenie) [13]: ticket lifecycle, escalation, notification, SLA tracking and audit trail. Gap: they are designed for IT and DevOps, with no IoT field-event ingestion and no outdoor or trekking domain context.

Gap statement: TrekLink combines five dimensions in one domain-specific platform: off-grid LoRa mesh communication, store-and-forward synchronization under intermittent connectivity, device rental lifecycle management, real-time field monitoring, and structured SOS-to-incident response.

Key References:

- [1] A. W.-L. Wong, S. L. Goh, M. K. Hasan, and S. Fattah, “Multi-hop and mesh for LoRa networks: Recent advancements, issues, and recommended applications,” ACM Comput. Surv., vol. 56, no. 6, Article 136, pp. 1–43, Jan. 2024. DOI: 10.1145/3638241
- [2] Muladi, H. Wijaya, S. D. Prasetyo, S. A. Hamzah, and A. K. Mahamad, “LoRa mesh-based IoT GPS tracking system for mountain climbers,” Int. J. Safety Security Eng., vol. 14, no. 6, pp. 1751–1761, Dec. 2024. DOI: 10.18280/ijsse.140610

- [3] H. Zhang, R. Zhang, and J. Sun, “Developing real-time IoT-based public safety alert and emergency response systems,” Sci. Rep., vol. 15, Article 29056, Aug. 2025. DOI: 10.1038/s41598-025-13465-7
- [4] OASIS, “MQTT Version 5.0,” OASIS Standard, Mar. 2019. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- [5] V. Cerf, S. Burleigh, A. Hooke, L. Torgerson, R. Durst, K. Scott, K. Fall, and H. Weiss, “Delay-tolerant networking architecture,” IETF RFC 4838, Apr. 2007. https://www.rfc-editor.org/rfc/rfc4838
- [6] Meshtastic Project, “Official Documentation,” 2024. [Online]. Available: https://meshtastic.org/docs/
- [7] W. Vogels, “Eventually consistent,” Commun. ACM, vol. 52, no. 1, pp. 40–44, Jan. 2009. DOI: 10.1145/1435417.1435432
- [8] D. Harel, “Statecharts: A visual formalism for complex systems,” Sci. Comput. Program., vol. 8, no. 3, pp. 231–274, Jun. 1987. DOI: 10.1016/0167-6423(87)90035-9
- [9] P. Helland, “Idempotence is not a medical condition,” Commun. ACM, vol. 55, no. 5, pp. 56–65, May 2012. DOI: 10.1145/2160718.2160734
- [10] A. Stanford-Clark and H. L. Truong, “MQTT For Sensor Networks (MQTT-SN) Protocol Specification, Version 1.2,” IBM, Nov. 2013. [Online]. Available: https://www.oasis-open.org/committees/download.php/66091/MQTT-SN_spec_v1.2.pdf
- [11] LoRa Alliance, “TS001-1.0.4 LoRaWAN L2 1.0.4 Specification.” [Online]. Available: https://resources.lora-alliance.org/technical-specifications/ts001-1-0-4-lorawan-l2-1-0-4-specification
- [12] Amazon Web Services, “What is AWS IoT?,” AWS IoT Core Developer Guide. [Online]. Available: https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html
- [13] PagerDuty, “Escalation Policy Basics,” PagerDuty Knowledge Base. [Online]. Available: https://support.pagerduty.com/main/docs/escalation-policies
- [14] Garmin Ltd., “inReach Mini 2 satellite communicator,” product page. [Online]. Available: https://www.garmin.com/en-US/p/837461/pn/010-02602-00

4. Other comments (propose all relative things if have):

The TrekLink LoRa mesh firmware and hardware (ESP32/ESP32-S3) came out of this team’s Summer 2026 project and are the technical foundation of this Capstone. The targeted firmware enhancements made during the Capstone are in scope and credited to it. The reports document the hardware design work.

Supervisor (If have) ***(Sign and full name)***	**HCM, date ____/____/2026** **On behalf of Registers** ***(Sign and full name)***